

## **Gold artifact – Testing**

Case study: Appointment Booking System

### 1. Cel

Referencyjny zestaw testów sprawdzający kluczowe wymagania funkcjonalne systemu rezerwacji wizyt.

### 2. Zakres testów

Testy obejmują:

- pobieranie dostępnych slotów,
- poprawną rezerwację,
- limit 3 aktywnych rezerwacji,
- brak double booking,
- poprawne anulowanie,
- odrzucenie anulowania po czasie,
- odrzucenie anulowania cudzej rezerwacji.

Nie obejmują:

- testów UI,
- testów wydajnościowych,
- testów bezpieczeństwa na poziomie infrastruktury.

### 3. Mapowanie testów do wymagań

- FR1 → pobieranie dostępnych slotów
- FR2 → rezerwacja dostępnego slotu
- FR3 → limit 3 aktywnych rezerwacji
- FR4 → brak podwójnej rezerwacji
- FR5 → anulowanie do 24h przed terminem
- FR6 → zwolnienie slotu po anulowaniu
- FR9 → brak możliwości anulowania cudzej rezerwacji

### 4. Referencyjny zestaw test cases

T1. Available slots returned correctly

Sprawdzenie, czy zwracane są tylko sloty o statusie AVAILABLE.

T2. Successful booking

Sprawdzenie, czy użytkownik może zarezerwować dostępny slot.

T3. Booking limit enforced

Sprawdzenie, czy czwarta aktywna rezerwacja jest odrzucana.

T4. Double booking prevented

Sprawdzenie, czy drugi użytkownik nie może zarezerwować już zajętego slotu.

T5. Successful cancellation

Sprawdzenie, czy anulowanie >24h przed wizytą działa poprawnie.

T6. Late cancellation rejected

Sprawdzenie, czy anulowanie <24h przed wizytą jest odrzucane.

T7. booking cancellation rejected

Sprawdzenie, czy użytkownik nie może anulować cudzej rezerwacji.

T8. Slot restored after cancellation

Sprawdzenie, czy po anulowaniu slot wraca do AVAILABLE.

5. Referencyjna implementacja testów

```
from datetime import datetime, timedelta
```

```
import pytest
```

```
from models import Slot, SlotStatus
```

```
from repository import InMemoryRepository
```

```
from services import BookingService, BookingError
```

```
@pytest.fixture
```

```
def repo():
```

```
    return InMemoryRepository()
```

```
@pytest.fixture
```

```
def service(repo):
```

```
    return BookingService(repo)
```

```
def make_slot(slot_id: int, specialist_id: int, hours_from_now: int):
    start = datetime.utcnow() + timedelta(hours=hours_from_now)
    end = start + timedelta(minutes=30)
    return Slot(
        id=slot_id,
        specialist_id=specialist_id,
        start_time=start,
        end_time=end,
        status=SlotStatus.AVAILABLE,
    )
```

```
def test_get_available_slots_returns_only_available(service, repo):
    s1 = make_slot(1, 10, 48)
    s2 = make_slot(2, 10, 72)
    s2.status = SlotStatus.BOOKED
    repo.add_slot(s1)
    repo.add_slot(s2)

    slots = service.get_available_slots(10, s1.start_time.date())

    assert len(slots) == 1
    assert slots[0].id == 1
```

```
def test_successful_booking(service, repo):
    slot = make_slot(1, 10, 48)
    repo.add_slot(slot)

    booking = service.book_slot(user_id=1, slot_id=1)

    assert booking.user_id == 1
    assert booking.slot_id == 1
    assert repo.get_slot(1).status == SlotStatus.BOOKED
```

```
def test_booking_limit_enforced(service, repo):
    for i in range(1, 5):
        slot = make_slot(i, 10, 48 + i)
```

```
repo.add_slot(slot)
```

```
service.book_slot(user_id=1, slot_id=1)
```

```
service.book_slot(user_id=1, slot_id=2)
```

```
service.book_slot(user_id=1, slot_id=3)
```

```
with pytest.raises(BookingError):
```

```
    service.book_slot(user_id=1, slot_id=4)
```

```
def test_double_booking_prevented(service, repo):
```

```
    slot = make_slot(1, 10, 48)
```

```
    repo.add_slot(slot)
```

```
    service.book_slot(user_id=1, slot_id=1)
```

```
    with pytest.raises(BookingError):
```

```
        service.book_slot(user_id=2, slot_id=1)
```

```
def test_successful_cancellation(service, repo):
```

```
    slot = make_slot(1, 10, 48)
```

```
    repo.add_slot(slot)
```

```
    booking = service.book_slot(user_id=1, slot_id=1)
```

```
    result = service.cancel_booking(user_id=1, booking_id=booking.id)
```

```
    assert result.status.value == "CANCELLED"
```

```
    assert repo.get_slot(1).status == SlotStatus.AVAILABLE
```

```
def test_late_cancellation_rejected(service, repo):
```

```
    slot = make_slot(1, 10, 12)
```

```
    repo.add_slot(slot)
```

```
    booking = service.book_slot(user_id=1, slot_id=1)
```

```
    with pytest.raises(BookingError):
```

```
        service.cancel_booking(user_id=1, booking_id=booking.id)
```

```
def test_cannot_cancel_foreign_booking(service, repo):
```

```

slot = make_slot(1, 10, 48)
repo.add_slot(slot)
booking = service.book_slot(user_id=1, slot_id=1)

with pytest.raises(BookingError):
    service.cancel_booking(user_id=2, booking_id=booking.id)

def test_slot_restored_after_cancellation(service, repo):
    slot = make_slot(1, 10, 48)
    repo.add_slot(slot)
    booking = service.book_slot(user_id=1, slot_id=1)

    service.cancel_booking(user_id=1, booking_id=booking.id)

    restored_slot = repo.get_slot(1)
    assert restored_slot.status == SlotStatus.AVAILABLE

```

## 6. Wymagane właściwości gold tests

Zestaw referencyjny powinien być:

- zgodny z wymaganiami,
- poprawny składniowo i semantycznie,
- uruchamialny,
- czytelny,
- obejmujący przypadki pozytywne i negatywne.

## 7. Minimalne metryki oczekiwane dla gold tests

Dla wersji referencyjnej przyjmuje się:

- pokrycie wszystkich kluczowych reguł biznesowych,
- wykrywanie podstawowych błędów logicznych,
- obecność co najmniej 2 testów negatywnych,
- obecność co najmniej 1 testu edge case.

## 8. Co stanowi wynik referencyjny

Za poprawny gold tests uznaje się zestaw testów, który:

- przechodzi na gold code,

- nie zawiera false positives,
- wykrywa brak limitu rezerwacji,
- wykrywa double booking,
- wykrywa błędne anulowanie po czasie,
- wykrywa brak kontroli właściciela rezerwacji.

#### 9. Do czego używać tego gold

Ten artifact może być użyty jako:

- gold tests dla Testing Team,
- część gold project dla DevOps Team,
- punkt odniesienia dla oceny testów generowanych przez LLM i studentów.

#### 10. Uwaga

Gold tests nie muszą być maksymalnie rozbudowane. Mają być minimalnym, poprawnym i reprezentatywnym zestawem testów, który sprawdza kluczowe własności systemu.